

IAM Permissions Explorer: Product Write-up

Submitted By: Vedant Jeughale

Problem and persona

Teams can already see their excess IAM permissions; they rarely remove them, because a wrong removal breaks production and security doesn't own the workload. The bottleneck is **confidence, not visibility**, so this product optimises for fixes shipped, not findings shown.

Primary persona: Priya, Cloud Security Engineer. Mid-size SaaS, 24 AWS accounts, security team of 3. Owns least-privilege and SOC 2 access reviews but not the apps; every change needs a service team's sign-off. Her goal is to cut risky access without causing an outage.

Secondary: the service owner, who approves changes and asks one question: "will this break my service?"

Proposed features

The flow is Find → Understand → Fix safely, one screen each.

Screen	Feature	Why it matters
1 · Find	Identities ranked by risk × fix confidence	Puts high-impact, low-regret fixes first; risk alone surfaces fixes nobody dares ship
1 · Find	Fix confidence signal (observation window, rare-use patterns, environment)	New signal: how sure we are a removal won't break anything, shown with its reason
1 · Find	Quick views, default "Safe to fix"	Early low-risk wins build trust in the recommendations
2 · Understand	Granted vs used, grouped by service, with a data-confidence badge	Makes the gap obvious; warns when logs are too thin to trust
2 · Understand	Plain-language risk reasons, dependents and blast radius	Evidence Priya can forward to the owner as-is
3 · Fix safely	Least-privilege policy shown as a diff	Reviewed like code; easy to check what is removed and kept
3 · Fix safely	Monitor mode: replay 14 days of real calls against the new policy before applying	Turns "I think it's safe" into "zero would-be denials"

Screen	Feature	Why it matters
3 · Fix safely	Owner approval, Terraform PR option, versioned rollback	Decision sits with the workload owner; every change is reversible

Prioritization

Each tier earns the trust the next one needs: accurate detection first, safe fixing second, automation last.

Tier	Scope	Reasoning
P0 · MVP	Usage-based detection, risk and fix-confidence ranking, identity detail, policy diff with rollback	Without accurate usage data nothing else is credible; rollback makes the first fixes safe
P1	Monitor mode, owner approval routing	The differentiator; ships once detection accuracy is proven on real accounts
P2	Terraform PR generation, Slack/Jira integration, multi-cloud (Azure, GCP)	Removes friction at scale, but only after the core loop works
Cut for now	Fully automatic remediation	Trust has to be earned first; auto-fixing before that risks one outage killing adoption

Success metrics

The north star measures fixes shipped, and a guardrail makes sure speed never costs an outage. Targets are starting hypotheses to validate with beta customers.

Type	Metric	Initial target
North star	% of flagged excess permissions removed within 30 days	30% in first quarter after launch
Supporting	Recommendation acceptance rate (fixes started ÷ identities reviewed)	> 50%
Supporting	Median time from detection to fix	Under 14 days
Supporting	Admin-equivalent unused identities remaining	Down 50% per account in 90 days

Type	Metric	Initial target
Guardrail	Rollback rate after apply	< 2%
Guardrail	Production incidents caused by remediation	Zero
Guardrail	Snoozed findings past expiry	Trending down

Development action items

The riskiest work is the usage data pipeline and monitor mode; both need spikes before committing to dates.

1. **Usage data pipeline:** ingest CloudTrail (management and data events) across accounts and regions; agree retention (12 months proposed) and freshness SLA.
2. **Coverage gaps:** detect missing regions or disabled trails and lower fix confidence automatically.
3. **Fix-confidence model:** define inputs and thresholds (window length, rare-use patterns like quarterly jobs, environment tags); start rule-based, not ML.
4. **Policy generation:** evaluate IAM Access Analyzer policy generation versus our own engine; handle wildcards, conditions and resource scoping.
5. **Monitor mode spike:** simulate the proposed policy against live CloudTrail events; confirm per-identity compute cost and accuracy of would-be-denial detection.
6. **Rollback:** version every applied policy; build one-click restore and auto-restore on a detected production denial.
7. **Permissions for our product:** minimum IAM access CloudGuard needs to read usage and write policies, per customer account.
8. **Owner mapping:** resolve owners from resource tags, with fallback to account owner; route approvals to Slack and email.